

# Práctica Tema 4

Despliegue de Aplicaciones Web  
María Pérez Ureña  
Daniel Suárez Martín  
15 de marzo de 2026

# Índice

|                                        |    |
|----------------------------------------|----|
| Introducción.....                      | 3  |
| Análisis del sistema.....              | 4  |
| Diseño de la base de datos.....        | 5  |
| Arquitectura de la aplicación.....     | 7  |
| Implementación de funcionalidades..... | 9  |
| Interfaz del usuario.....              | 14 |
| Conclusión.....                        | 15 |
| Bibliografía.....                      | 16 |
| Anexos.....                            | 17 |

# Introducción

Esta última práctica de la asignatura tiene como objetivo desarrollar una aplicación web dinámica para gestionar los gastos domésticos de los usuarios. Para su implementación, se han usado tecnologías propias del desarrollo de aplicaciones web en Java, tales como Servlets, JSP, acceso a bases de datos mediante MySQL y el uso de pool de conexiones.

Esta aplicación permite a los usuarios registrados acceder mediante un sistema de autenticación y gestionar las compras realizadas en distintas tiendas. Entre las funcionalidades principales encontramos el registro, consulta, modificación y eliminación tanto de las compras como de las tiendas donde se realizan.

El sistema, además, proporciona una vista inicial con las compras realizadas durante el mes actual, ordenadas cronológicamente, junto con el cálculo del gasto total mensual. También cuenta con un apartado de informes para consultar el historial de compras organizado en función de los meses disponibles en la base de datos.

Para el desarrollo de la aplicación se ha seguido el patrón de arquitectura Modelo-Vista-Controlador (MVC), separando claramente la lógica de negocio, la gestión de datos y la interfaz de usuario. De esta forma se consigue una aplicación más organizada, mantenible y escalable.

El objetivo de esta práctica es integrar y aplicar los conocimientos adquiridos sobre desarrollo web con Java, bases de datos relacionales, acceso a datos mediante DAO, y buenas prácticas de diseño de software en aplicaciones web.

# Análisis del sistema

El objetivo principal del sistema es ofrecer una herramienta sencilla e intuitiva que nos permita llevar un control de los gastos personales realizados en diferentes tiendas a lo largo de los meses. El sistema está orientado a registrar compras realizadas por el usuario, indicando la fecha de la compra, el importe gastado y la tienda donde se ha realizado.

Para ello, la aplicación permite gestionar dos elementos principales: las tiendas y las compras. Cada compra está asociada a una única tienda, mientras que una misma tienda puede tener múltiples compras registradas.

El sistema también incorpora un mecanismo de autenticación para controlar el acceso a la aplicación. De esta forma, solamente los usuarios registrados en la base de datos pueden acceder a las funcionalidades del sistema.

Entre las funcionalidades principales del sistema destacan:

- Inicio de sesión mediante usuario y contraseña.
- Visualización de las compras realizadas durante el mes actual.
- Cálculo automático del gasto total del mes.
- Registro de nuevas compras.
- Edición de compras existentes.
- Eliminación de compras con confirmación previa.
- Gestión de tiendas (alta, edición y borrado).
- Generación de informes de compras por meses disponibles en la base de datos.
- Cierre de sesión del usuario.

Estas funcionalidades permiten mantener un control organizado de los gastos realizados y consultar información histórica almacenada en el sistema.

# Diseño de la base de datos

El diseño de la base de datos se ha realizado utilizando un modelo relacional que permite almacenar la información necesaria para el funcionamiento de la aplicación.

La base de datos se compone de tres tablas principales: usuarios, tiendas y compras.

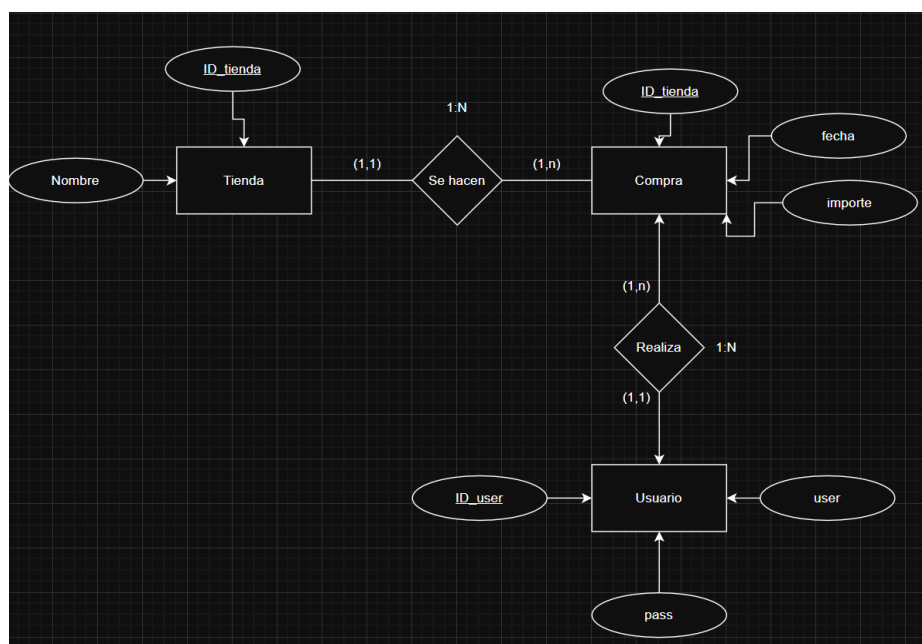
## Diagrama Entidad-Relación

En el modelo Entidad-Relación se representan las entidades principales del sistema y las relaciones existentes entre ellas.

Las entidades identificadas son:

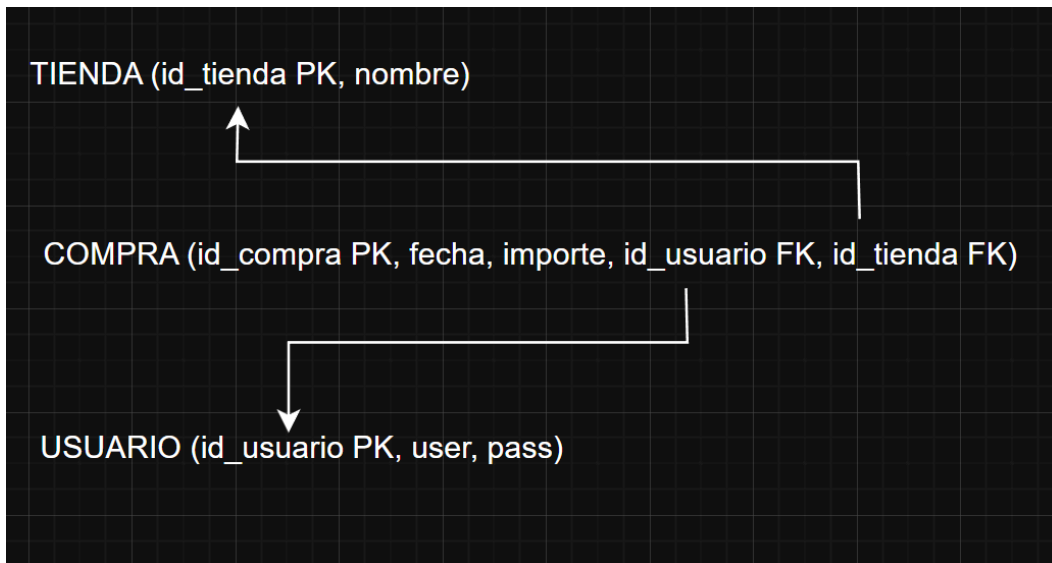
- Usuario, que almacena los datos necesarios para la autenticación en el sistema.
- Tienda, que representa los establecimientos donde se realizan las compras.
- Compra, que registra cada gasto realizado por el usuario.

La relación existente entre tiendas y compras es de tipo uno a muchos (1:N), ya que una tienda puede tener asociadas múltiples compras, pero cada compra solamente puede pertenecer a una única tienda. De la misma manera, la relación entre usuarios y compras también es uno a muchos, pues cada usuario podrá tener asociado a su cuenta varias compras, mientras que cada una de estas deberá estar asociada al perfil de una sola persona.



## Esquema Relacional

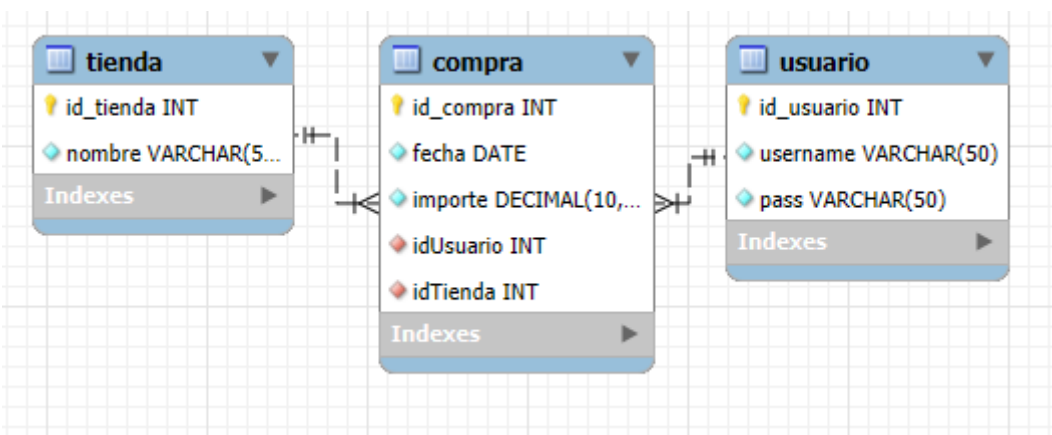
A partir del modelo Entidad-Relación anterior, se obtiene el esquema relacional de la base de datos.



Como se puede observar en el mismo, cada tabla tendrá como clave primaria un ID representativo, además de los distintos elementos necesarios para cada una. Por su parte, la tabla de compras (COMPRA), tendrá asociada como claves foráneas estos mismos ID tanto de los usuarios como de las tiendas para, de esta forma, generar la relaciones necesarias para las identificaciones en la base de datos.

## Modelo MySQL en Workbench

Una vez definido el esquema relacional, se ha creado el modelo de la base de datos utilizando la herramienta MySQL Workbench, que permite visualizar gráficamente las tablas y sus relaciones.



Además, como se puede observar en el Anexo I, las relaciones de compra con usuario y tienda se han realizado de tal forma que no permita el borrado de elementos que tienen datos asociados.

# Arquitectura de la aplicación

La aplicación se ha desarrollado siguiendo el patrón arquitectónico Modelo-Vista-Controlador (MVC), el cual permite separar las diferentes responsabilidades del sistema en tres componentes principales.

Esta organización facilita el mantenimiento del código, mejora su reutilización y permite una mayor claridad en la estructura del proyecto.

## Modelo

El modelo se encargará de dar forma y configurar la lógica de negocio y el acceso a los datos almacenados en la base de datos.

En esta capa se han desarrollado diferentes tipos de clases:

- **Clases POJO:** Estas representan las entidades de la base de datos mediante objetos Java. En el proyecto contamos con Usuario, Tienda y Compra. Estas clases tienen los atributos correspondientes a las columnas de las tablas, además de los métodos getters y setters necesarios para acceder y modificar sus valores.
- **Clases DAO:** Estas se encargan de realizar las operaciones de acceso a la base de datos. Entre las distintas funciones que tienen, encontramos la inserción de registro, consulta de datos, actualización de información, eliminación de registros... Se ha decidido optar por realizar solo los DAO de Tienda y Compra, pues esta aplicación no realiza ningún tipo de gestión directa con los usuarios (añadir o eliminarlos). Gracias a estas clases podemos encapsular las consultas SQL necesarias a través de distintos métodos.
- **Pool de conexiones:** Necesarias para gestionar las conexiones con la base de datos de forma más ágil y sin generar saturación en el tráfico de datos. Nos permite reutilizar conexiones ya abiertas en lugar de crear una nueva con cada consulta. Hemos optado, en este caso, por separar las clases de Login y Logout en lugar de aunarlas en un mismo archivo para una mayor claridad y facilidad a la hora de realizar modificaciones y encontrar errores de lógica del programa.

## Vista

La capa de vista está compuesta por las páginas JSP que se encargan de mostrar la información al usuario y recoger los datos introducidos en los formularios. Entre las vistas desarrolladas en la aplicación se encuentran: página de inicio de sesión, pantalla principal con listado de compras, listado de informes de compras filtrado por meses, formulario de alta de compra, formulario de edición de compra, listado de tiendas, formulario de alta de tiendas y formulario de edición de tiendas.

En estas páginas también se han utilizado elementos de HTML, CSS y JavaScript para mejorar la presentación de la información y la interacción con el usuario.

Además, para mostrar mensajes de confirmación y error a la hora de acceder a la aplicación o cerrar sesión, se ha utilizado la librería Alertify, que permite mostrar alertas con un diseño más moderno que las alertas estándar de JavaScript.

## Controlador

Formado por Servlets que se encargan de gestionar las peticiones realizadas por el usuario y coordinar la comunicación entre el modelo y la vista.

Cuando el usuario realiza una acción en la interfaz, como enviar un formulario o seleccionar una opción del menú, la petición es enviada a un servlet que procesa la solicitud y ejecuta la lógica correspondiente.

Entre las tareas realizadas por los controladores se encuentran:

- Validar los datos introducidos por el usuario.
- Llamar a los métodos de las clases DAO para acceder a la base de datos.
- Enviar los resultados a las páginas JSP para su visualización.
- Gestionar redirecciones entre páginas.

En este caso, hemos optado por la realización de dos archivos de controlador, uno de compras y otro de tiendas, pues nuevamente, no se llevan a cabo gestiones con la creación ni modificación de usuarios. Además, en pos de utilizar los distintos tipos de comunicación que permiten los Servlets, tanto mediante métodos *post* como *get*, cada uno de ellos trabaja en una de estas modalidades.



# Implementación de funcionalidades

En este apartado vamos a describir las funcionalidades principales implementadas en nuestra aplicación, dividiéndola en: sistema de autenticación, gestión de compras y gestión de tiendas.

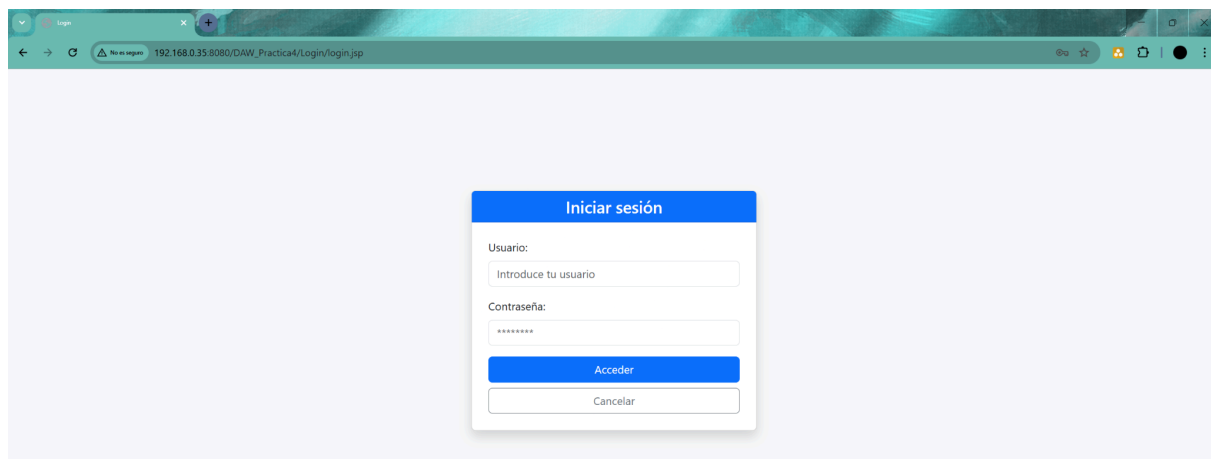
## Sistema de autenticación

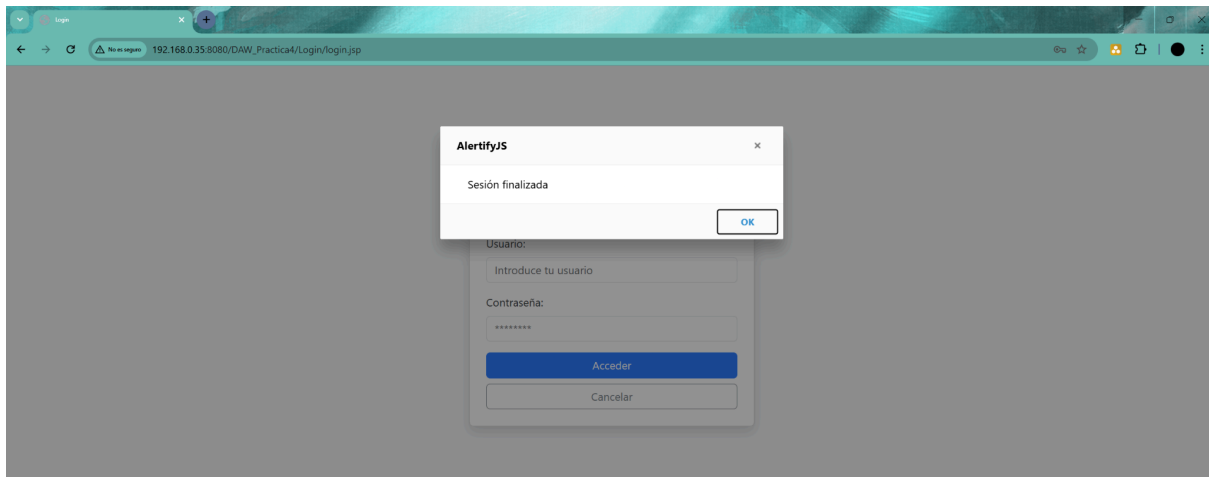
El acceso a la aplicación está protegido mediante un sistema de autenticación basado en usuario y contraseña.

Cuando el usuario introduce sus credenciales en el formulario de login, la aplicación consulta la base de datos para comprobar si existe un registro que coincida con los datos introducidos. Si la autenticación es correcta, se crea una sesión de usuario y se redirige a la pantalla principal.

En caso contrario, se muestra un mensaje de error mediante una alerta de la librería Alertify.

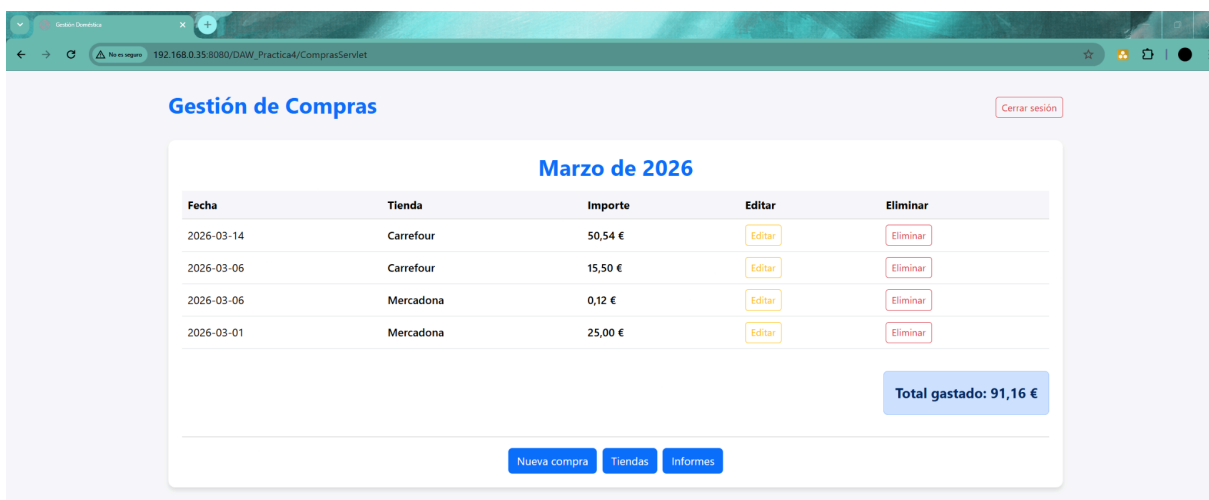
También se ha implementado un mecanismo de logout que permite cerrar la sesión del usuario y redirigir nuevamente a la página de inicio de sesión. Cuando esto sucede, nuevamente se vuelve a lanzar una alerta.





## Gestión de compras

La gestión de compras constituye una de las funcionalidades principales de la aplicación. Desde la pantalla principal el usuario puede visualizar todas las compras realizadas durante el mes actual, ordenadas de la más reciente a la más antigua.



Además de esto, se muestra el total del importe gastado durante el mes.

Este sistema permite realizar las siguientes operaciones:

- **Alta de compras.** Al utilizar esta función, se nos redirige a una página distinta donde, mediante un formulario, se nos pide introducir la fecha (por defecto se asocia el día actual), el importe y la tienda asociada de entre un desplegable de opciones con las tiendas existentes en la base de datos.

- **Edición de compras.** Al igual que la función de alta, al utilizarse este botón se nos envía a una página distinta donde se nos permite modificar los datos de una compra existente, los cuales aparecerán por defecto como opciones a elegir.

- **Eliminación de compras.** Al usar esta opción se nos permite eliminar una compra de nuestra base de datos, para lo cual deberemos confirmar ante el aviso emergente.

192.168.0.35:8080 dice  
¿Eliminar esta compra?

Aceptar Cancelar

**Gestión de Compras**

Marzo de 2026

| Fecha      | Tienda    | Importe | Editar                 | Eliminar               |
|------------|-----------|---------|------------------------|------------------------|
| 2026-03-14 | Carrefour | 50,54 € | <a href="#">Editar</a> | <a href="#">Borrar</a> |
| 2026-03-06 | Carrefour | 15,50 € | <a href="#">Editar</a> | <a href="#">Borrar</a> |
| 2026-03-06 | Mercadona | 0,12 €  | <a href="#">Editar</a> | <a href="#">Borrar</a> |
| 2026-03-01 | Mercadona | 25,00 € | <a href="#">Editar</a> | <a href="#">Borrar</a> |

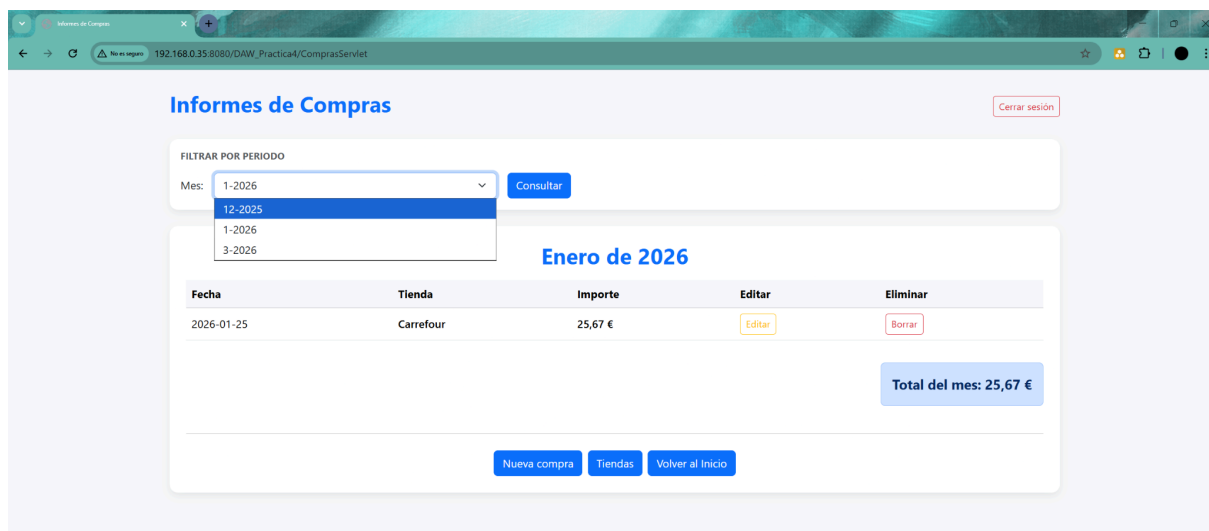
Total gastado: 91,16 €

[Nueva compra](#) [Tiendas](#) [Informes](#)

## Informes

De forma paralela a la gestión de compras, nos encontramos con la página de Informes, la cual funciona de una forma muy similar. Aquí también tendremos la opción de añadir, eliminar o editar compras, las cuales veremos ordenadas por fecha, y tendremos un sumatorio total de lo gastado.

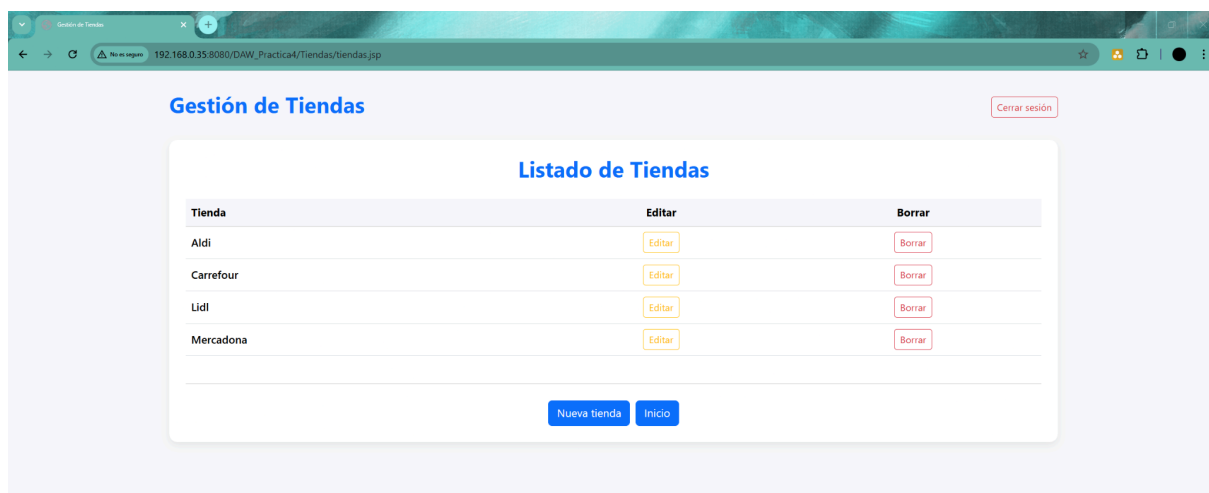
La única diferencia con la parte de gestión de compras es que aquí podremos filtrar las compras realizadas por el usuario en función a los meses donde encontramos compras realizadas.



Cada vez que el usuario seleccione un mes distinto, la tabla se actualizará con los datos de compra de ese periodo.

## Gestión de tiendas

Por último, la aplicación también permite gestionar las tiendas donde se realizan las compras.



Las operaciones disponibles son:

- **Alta de nuevas tiendas.**

A screenshot of a web browser showing a form titled "Añadir una tienda nueva". The form has a blue header bar with the title. Below the header, there is a label "Nombre de la tienda:" followed by a text input field containing the placeholder text "Ej: Mercadona, Amazon...". Below the input field, there are three buttons: a blue "Añadir Tienda" button, a white "Limpiar formulario" button, and a blue "Volver al listado" button.

- **Modificación del nombre de una tienda existente.**

A screenshot of a web browser showing a form titled "Editar tienda: Aldi". The form has a yellow header bar with the title. Below the header, there is a label "Nuevo nombre de la tienda:" followed by a text input field containing the text "Aldi". Below the input field, there are two buttons: a yellow "Guardar Cambios" button and a blue "Cancelar y volver" button.

- **Eliminación de tiendas.**

A screenshot of a web browser showing the "Gestión de Tiendas" page. The page has a dark blue header with the title "Gestión de Tiendas". Below the header, there is a confirmation dialog box with the text "192.168.0.35:8080 dice: ¿Eliminar esta tienda? Si la tienda tiene compras asociadas, no será posible eliminarla". The dialog has "Aceptar" and "Cancelar" buttons. Below the dialog, there is a table titled "Listado de Tiendas" with three columns: "Tienda", "Editar", and "Borrar". The table lists four stores: Aldi, Carrefour, Lidl, and Mercadona. Each store has an "Editar" button and a "Borrar" button. At the bottom of the table, there are two buttons: "Nueva tienda" and "Inicio".

| Tienda    | Editar                 | Borrar                 |
|-----------|------------------------|------------------------|
| Aldi      | <a href="#">Editar</a> | <a href="#">Borrar</a> |
| Carrefour | <a href="#">Editar</a> | <a href="#">Borrar</a> |
| Lidl      | <a href="#">Editar</a> | <a href="#">Borrar</a> |
| Mercadona | <a href="#">Editar</a> | <a href="#">Borrar</a> |

Si bien se trata de un sistema muy similar al de gestión de compras, en este caso encontramos una diferencia fundamental: el sistema impide eliminar una tienda si existen compras asociadas a ella, garantizando así la integridad de los datos almacenados en la base de datos.

# Interfaz del usuario

Una vez comentadas las funcionalidades de la aplicación y su estructura interna, toca hablar de la parte visible de la misma: la interfaz de usuario. Esta se ha diseñado con el objetivo de ofrecer una navegación sencilla y clara para el usuario.

Las distintas pantallas de la aplicación permiten acceder fácilmente a las funcionalidades principales, manteniendo una estructura visual coherente entre ellas.

Las alertas y confirmaciones se muestran con la finalidad de aportar información extra al usuario, al mismo tiempo que para añadir una capa de seguridad extra a la hora de realizar acciones tan críticas como son eliminar una entrada de compras o tienda del sistema, mejorando la experiencia del usuario.

# Conclusión

El desarrollo de esta práctica nos ha permitido aplicar de forma integrada diversos conceptos relacionados con el desarrollo de aplicaciones web utilizando Java.

Durante la realización del proyecto se han puesto en práctica conocimientos sobre el uso de Servlets, páginas JSP, acceso a bases de datos mediante MySQL, la implementación del patrón de arquitectura MVC, así como el uso de las herramientas git para trabajar en conjunto a la hora de realizar un proyecto.

Además, se ha trabajado con técnicas de acceso a datos mediante el patrón DAO y se ha implementado un pool de conexiones para mejorar el rendimiento de la aplicación.

El resultado final es una aplicación funcional que permite gestionar de forma sencilla los gastos domésticos, ofreciendo herramientas para registrar compras, consultar información y generar informes.

Pensando de cara posibles mejoras futuras se podrían incorporar nuevas funcionalidades, como la generación de gráficos de gastos por meses, la exportación de informes en formato PDF o la posibilidad de gestionar diferentes usuarios con sus propios registros de compras.

# Bibliografía

- [Repositorio de Github.](#)



# Anexos

## Anexo I

De cara a mejorar la legibilidad y consistencia de la documentación, hemos decidido agregar el código utilizado para generar la base de datos en un anexo aparte. Además, para mejorar la seguridad del sistema, se ha creado un usuario específico con permisos únicamente sobre esta base de datos para evitar utilizar el usuario administrador del servidor de la misma. A continuación se adjunta el código MySQL completo:

### Creación y uso de la base de datos

```
CREATE DATABASE control_gastos;
```

```
USE control_gastos;
```

### Creación de las tablas con las distintas especificaciones

```
CREATE TABLE Usuario (
  id_usuario INT auto_increment primary key,
  username VARCHAR (50) NOT NULL UNIQUE,
  pass VARCHAR (50) NOT NULL
);
```

```
CREATE TABLE Tienda (
  id_tienda INT auto_increment primary key,
  nombre VARCHAR (50) NOT NULL
);
```

```
CREATE TABLE Compra (
  id_compra INT auto_increment primary key,
  fecha DATE NOT NULL,
  importe DECIMAL(10,2) NOT NULL,
  idUsuario INT NOT NULL,
  idTienda INT NOT NULL,
  CONSTRAINT fk_usuario FOREIGN KEY (idUsuario) REFERENCES Usuario
(id_usuario) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_tienda FOREIGN KEY (idTienda) REFERENCES Tienda (id_tienda) ON
DELETE RESTRICT ON UPDATE CASCADE
);
```

### Creación del usuario y consiguiente asignación de permisos

```
CREATE USER 'usuario_gastos'@'%' IDENTIFIED BY '12345';
```

```
GRANT ALL PRIVILEGES ON control_gastos.* TO 'usuario_gastos'@'%';
```

```
FLUSH PRIVILEGES;
```